Setup

How to run the system locally.

1. Prerequisites

Two things installed on the host:

- **Docker Desktop** (includes Docker Compose)
- **An LLM API key** — OpenAI GPT-4o. Get one at https://platform.openai.com/api-keys.

That's the complete list. No local Python, no local Node, no local Postgres — everything runs in containers.

2. Three-command quickstart

```
git clone https://github.com/pradipkundu144/intelligent-sql-agent
cd sql-agent
cp .env.example .env      # then open .env and paste your LLM_API_KEY
docker compose up
```

Wait ≈90 seconds on the first run. Docker pulls the four images (≈200 MB total), the Postgres container seeds 500 customers / 100 products / 800 orders / ≈2,000 order items, and the backend startup ingests the RAG examples into pgvector.

Subsequent runs boot in ≈10 seconds.

3. Where to open

URL	WHAT IT IS
http://localhost:5173	**Chat interface** — the primary user-facing UI
http://localhost:5173/dashboard	**Admin dashboard** — streaming evaluation + interactive architecture visualisation
http://localhost:8000/docs	**API docs** — FastAPI Swagger UI
http://localhost:8000/health	**Health check** — reports Postgres + MongoDB reachability

Click the "Dashboard ↗" button in the chat header to open the dashboard in a new tab.

4. Verifying the install

A reviewer can confirm the stack works in under a minute:

CHECK	HOW	EXPECTED
App loads	Open http://localhost:5173	Chat UI with three example chips visible
Happy path	Click "How many customers are there?"	Plain-English answer (500) + collapsible SQL panel + trace panel
Safety layer	Type "delete all customers" and submit	Calm red block message, no execution, distinct destructive-pill label
Multi-month data	Click "Revenue by month"	Multiple rows returned, not a single month
Observability	Expand the trace panel on any answer	Per-stage latency bars, tokens, cost, "View full trace in Langfuse" link
Multi-intent	Try "How many customers, and revenue by month?"	Two side-by-side answer cards, each with its own SQL and trace

5. Running the evaluation

Two ways to reproduce every number in 04_EVALUATION :

Via CLI:

```
docker compose exec backend python -m eval.run
```

Prints a pass-rate table; writes full results to `eval/results/latest.json`.

Via dashboard: Open http://localhost:5173/dashboard → Evaluation tab → click "Run evaluation". Per-case results stream live; a summary modal auto-opens on completion.

Full eval takes ≈2–3 minutes (RAGAS scoring dominates).

6. Environment configuration

All configuration lives in `.env` (never committed) and `.env.example` (template, safe to commit).

Required:

- `LLM_API_KEY` — the only value the reviewer must supply

Defaults that work as-is for local:

- `LLM_MODEL=gpt-4o`, `EMBEDDING_MODEL=text-embedding-3-small`
- `POSTGRES_USER=app`, `POSTGRES_PASSWORD=app`, `POSTGRES_DB=shop_db`
- `MONGO_URI=mongodb://mongodb:27017/sql_agent`
- `SAFE_INLINE_LIMIT=250`, `HARD_ROW_CEILING=50000`, `OVERFLOW_SAMPLE_SIZE=20`
- `QUERY_TIMEOUT_MS=5000`, `MAX_RETRIES=3`, `MAX_SUBQUESTIONS=5`

Optional — degrade gracefully if absent:

- `LANGFUSE_PUBLIC_KEY`, `LANGFUSE_SECRET_KEY`, `LANGFUSE_HOST`, `LANGFUSE_PROJECT_ID` — for Langfuse Cloud trace URLs
- `LANGCHAIN_TRACING_V2=true`, `LANGCHAIN_API_KEY`, `LANGCHAIN_PROJECT` — for LangSmith (auto-detected by `langchain-core`)
- `RAG_ENABLED=true` — feature flag for the RAG layer (set `false` to reproduce the RAG A/B in 04_EVALUATION §3)

7. Startup ordering

Docker Compose handles boot order via healthchecks:

- `postgres` is ready when `pg_isready` succeeds (first boot: ≈20s including init.sql + seed.sql; subsequent boots: ≈3s)
- `mongodb` is ready when `db.adminCommand('ping')` returns `ok: 1`
- `backend` waits for **both** databases to pass their healthchecks before starting
- `frontend` starts after `backend` (no health condition — Vite dev server retries on connection errors)

8. Reset the seed data

Postgres init scripts (`db/init.sql`, `db/seed.sql`) only run against an **empty volume**. To force a reseed:

```
docker compose down -v && docker compose up
```

The `-v` flag deletes the named volumes (`postgres_data`, `mongo_data`). Without it, seed changes have no effect on subsequent boots.

9. Troubleshooting

SYMPTOM	CAUSE	FIX
Backend logs "LLM_API_KEY missing"	Key not set in `.env`	Add the key, `docker compose up -d --force-recreate backend`
Port already in use	5173 / 8000 / 5432 / 27017 occupied	Stop the conflicting service or remap the port in `docker-compose.yml`
Backend can't reach DB on boot	Race condition on first boot	Handled by healthcheck; if seen, the retry resolves it automatically
Empty results everywhere	Seed didn't run	`docker compose down -v && docker compose up`
Schema/seed changes have no effect	Existing volume not wiped	`docker compose down -v` before re-boot
Env var changes not applied	`docker compose restart` doesn't reload `env_file`	`docker compose up -d --force-recreate backend`

10. Local fallback (no Docker)

If Docker isn't available, the stack can be run directly. Not the recommended path — it requires local Postgres (with pgvector extension), MongoDB, Python 3.11+, and Node 20+.

```
# database: a local Postgres with pgvector, then:
psql -f db/init.sql
psql -f db/seed.sql

# backend
cd backend && pip install -r requirements.txt
uvicorn app.main:app --reload --loop asyncio

# frontend
cd frontend && npm install && npm run dev
```

Docker is the recommended path precisely because it removes the version-mismatch and "works on my machine" risks this route reintroduces.

11. What Docker doesn't containerise

- **The hosted LLM (OpenAI)** — runs on OpenAI's infrastructure, reached over HTTPS. Requires a paid API key.
- **Langfuse Cloud** — external, reached only from the backend, uses public/secret key pair.
- **LangSmith** — external, same pattern.

A fully self-contained variant is possible by swapping the hosted model for a local open-source one (e.g. an Ollama container), removing the key requirement entirely. Not the default because hosted models produce materially better SQL and the demo prioritises generation quality.